

# Apex Nets Internship Documentation

## Android Development Fundamentals – Tutorial Notes

### Tutorial: Review Android Development Fundamentals

#### 1. Android Activities

An **Activity** is a single screen or entry point in an Android application. It provides the user interface and handles user interactions.

- Each Activity represents a screen or a specific task.
- Activities have a lifecycle managed by Android.
- Common lifecycle methods include `onCreate()`, `onStart()`, `onResume()`, `onPause()`, and `onDestroy()`.
- `onCreate()` is commonly used to initialize the screen and set its layout.
- Activities can start other Activities using **Intents**.
- An Activity can display a layout containing buttons, text, images, and other UI components.

#### Example

```
class LoginActivity : AppCompatActivity() {  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_login)  
    }  
}
```

Here:

- `LoginActivity` represents the login screen.
- `onCreate()` initializes the Activity.
- `setContentView()` connects the Activity with its XML layout.

#### 2. Android Fragments

A **Fragment** is a reusable portion of an Activity's user interface and behavior.

- A Fragment cannot normally work independently; it is hosted by an Activity.
- Fragments are useful for creating modular and reusable UI.
- Multiple Fragments can be used within one Activity.
- They are commonly used in applications with navigation, such as Home, Profile, and Settings.
- Fragments also have their own lifecycle.

#### Example

```
class HomeFragment : Fragment(R.layout.fragment_home) {  
  
    override fun onCreateView(  
        view: View,  
        savedInstanceState: Bundle?  
    ) {  
        super.onCreateView(view, savedInstanceState)  
    }  
}
```

For example, an application can use:

- HomeFragment → Home screen
- ProfileFragment → User profile
- SettingsFragment → Application settings

### 3. Android Layouts

A **Layout** defines how UI elements are arranged on an Android screen.

#### Common layouts include:

##### LinearLayout

Arranges views horizontally or vertically.

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome" />

</LinearLayout>
```

##### ConstraintLayout

Allows views to be positioned using constraints. It is commonly used to create flexible and responsive screens.

##### FrameLayout

Places views on top of each other and is commonly useful as a container for Fragments.

#### Important UI Components

- TextView – Displays text.
- EditText – Accepts user input.
- Button – Performs an action when clicked.
- ImageView – Displays images.
- RecyclerView – Displays lists of data efficiently.
- CardView – Creates card-style UI elements.

### 4. Activity, Fragment and Layout Relationship

**Activity → Fragment → Layout → UI Components**

```
MainActivity
  ↓
HomeFragment
  ↓
fragment_home.xml
  ↓
TextView + ImageView + Button
```

The Activity manages the screen, the Fragment provides a reusable section, and the Layout defines how the UI components are arranged.

### 5. Practical Example

Consider a **Student Management App**.

- MainActivity → Main container.
- HomeFragment → Displays student information.
- ProfileFragment → Displays student profile.
- fragment\_home.xml → Contains TextViews and Buttons.
- RecyclerView → Displays a list of students.

This structure makes the application easier to organize, maintain, and expand.

## **Key Takeaways**

- Activity manages a major screen or application entry point.
- Fragment provides reusable sections of an Activity.
- Layout defines the arrangement of UI elements.
- Understanding Activities, Fragments, and Layouts is essential for Android application development.
- Proper separation of screens and UI components makes an application easier to maintain.

# Tutorial: Android Layouts

## 1. Introduction

An **Android Layout** defines how user interface (UI) elements are arranged on the screen of an Android application.

Layouts are used to organize components such as:

- TextView
- EditText
- Button
- ImageView
- RecyclerView
- CheckBox
- RadioButton

Layouts can be created using **XML** or programmatically using Kotlin/Java. XML layouts are commonly used in traditional Android View-based development.

## 2. LinearLayout

LinearLayout arranges UI components in a single direction. It can arrange components vertically or horizontally. The direction is controlled using `android:orientation`.

### Example

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Username" />

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter username" />

    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Login" />

</LinearLayout>
```

### Use

LinearLayout is useful for simple forms, login screens, registration screens, and vertically or horizontally arranged components.

## 3. ConstraintLayout

ConstraintLayout allows UI elements to be positioned relative to the parent layout or other UI elements. It is useful for creating flexible and responsive interfaces.

### Example

```

<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Welcome"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

## Use

ConstraintLayout is commonly used for modern and responsive screen designs.

## 4. FrameLayout

FrameLayout is a simple container that allows views to be placed on top of each other. It is commonly used as a Fragment container, a placeholder for dynamic content, or an area where views overlap.

## Example

```

<FrameLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragment_container"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />

```

## Use

For example, MainActivity can use a FrameLayout as a container and display different Fragments inside it.

## 5. RelativeLayout

RelativeLayout positions UI elements relative to the parent or other views. For example, one TextView can be placed below another TextView.

## Example

```

<RelativeLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Student Profile" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Edit"
        android:layout_below="@id/title" />

</RelativeLayout>

```

## Use

RelativeLayout can be used when components need to be positioned relative to each other. However, ConstraintLayout is generally preferred for complex modern layouts.

## 6. ScrollView

ScrollView allows users to scroll vertically when the content is larger than the screen. It normally contains one direct child, such as a LinearLayout.

### Example

```
<ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- Content -->

    </LinearLayout>

</ScrollView>
```

### Use

It is useful for registration forms, profile pages, long information pages, and settings screens.

## 7. TableLayout

TableLayout arranges UI components in rows and columns. It is useful when information needs to be displayed in a table-like structure.

### Example

```
<TableLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">

    <TableRow>

        <TextView
            android:text="Name" />

        <TextView
            android:text="Parv" />

    </TableRow>

</TableLayout>
```

### Use

It can be used for simple tabular information such as student details or basic reports.

## 8. GridLayout

GridLayout arranges UI components in a grid consisting of rows and columns.

### Example

```
<GridLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:columnCount="2">

    <Button
        android:text="Home" />
```

```

        <Button
            android:text="Profile" />

        <Button
            android:text="Settings" />

        <Button
            android:text="Help" />

    </GridLayout>

```

## Use

GridLayout can be useful for dashboard screens, menu screens, category displays, and calculator-style interfaces.

## 9. CoordinatorLayout

CoordinatorLayout is a container designed to coordinate interactions between child views. It is commonly used with Material Design components such as AppBarLayout, FloatingActionButton, Snackbar, and CollapsingToolbarLayout.

## Example

```

<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- UI components -->

</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

## Use

It is useful for screens containing scrolling behavior, floating buttons, app bars, and Material Design interactions.

## 10. Common Layout Attributes

### layout\_width

Defines the width of a view. Common values: match\_parent – Uses the available parent width; wrap\_content – Uses only the space required by the content.

```

    android:layout_width="match_parent"

```

### layout\_height

Defines the height of a view.

```

    android:layout_height="wrap_content"

```

### padding

Adds space inside a view.

```

    android:padding="16dp"

```

### layout\_margin

Adds space outside a view.

```

    android:layout_margin="16dp"

```

### gravity

Controls the position of content inside a view.

```

    android:gravity="center"

```

### orientation

Defines the direction of a LinearLayout.

```
android:orientation="vertical"
```

## 11. Layout Selection Guide

| Layout            | Main Purpose                             |
|-------------------|------------------------------------------|
| LinearLayout      | Arrange views vertically or horizontally |
| ConstraintLayout  | Create flexible and responsive UI        |
| FrameLayout       | Container and overlapping views          |
| RelativeLayout    | Position views relative to each other    |
| ScrollView        | Display scrollable content               |
| TableLayout       | Arrange content in rows and columns      |
| GridLayout        | Arrange components in a grid             |
| CoordinatorLayout | Coordinate Material Design interactions  |

## 12. Practical Example – Login Screen

A login screen can be created using LinearLayout:

```
Login Screen
■
■■■ App Logo
■■■ Email EditText
■■■ Password EditText
■■■ Login Button
■■■ Create Account Button
```

The layout organizes these components vertically, while each UI component handles a specific user interaction.

## Key Takeaways

- Android layouts control the structure and positioning of UI elements.
- LinearLayout is suitable for simple horizontal or vertical arrangements.
- ConstraintLayout is useful for complex and responsive screens.
- FrameLayout is commonly used as a container for Fragments.
- ScrollView is useful when screen content is longer than the available display.
- Choosing the correct layout helps create organized, responsive, and maintainable Android applications.

# Tutorial: Android Intent and Toast

## Android Intent

### 1. Introduction

An **Intent** is a messaging object used in Android to request an action from another Android component.

Intents are commonly used to:

- Move from one Activity to another.
- Send data between Activities.
- Open another application or system feature.
- Start services or other Android components.

### 2. Types of Intent

#### Explicit Intent

An **Explicit Intent** specifies the exact Activity or component that should be opened.

#### Example

```
val intent = Intent(this, SecondActivity::class.java)
startActivity(intent)
```

Here, the application directly opens SecondActivity.

#### Practical Example

```
LoginActivity
  ↓
Login Button
  ↓
HomeActivity
```

An Explicit Intent can be used to navigate from the Login screen to the Home screen.

#### Implicit Intent

An **Implicit Intent** does not specify a particular application component. Instead, Android determines which application can perform the requested action.

#### Example – Open a Website

```
val intent = Intent(
    Intent.ACTION_VIEW,
    Uri.parse("https://www.google.com")
)
startActivity(intent)
```

Android will open an appropriate web browser.

#### Other Examples

Implicit Intents can be used to:

- Open a website.
- Make a phone call.
- Send an email.
- Open the camera.
- Share content.

- Open a location in a map application.

### 3. Passing Data Using Intent

Intent can also transfer data from one Activity to another.

#### Sending Data

```
val intent = Intent(this, ProfileActivity::class.java)
intent.putExtra("username", "Parv")
startActivity(intent)
```

#### Receiving Data

```
val username = intent.getStringExtra("username")
```

The value "Parv" is passed from one Activity to another.

### 4. Key Points of Intent

- Intent is used for communication between Android components.
- Explicit Intent is generally used for navigation within the same application.
- Implicit Intent is used when Android should find a suitable application to perform an action.
- Intent can transfer data using extras.
- startActivity() is commonly used to launch another Activity.

## Android Toast

### 1. Introduction

A **Toast** is a small temporary message displayed to the user.

It is useful for providing short feedback about an action without interrupting the user's current activity.

#### Common Uses

- Show a successful operation.
- Inform the user about an action.
- Display a short error message.
- Confirm that a button was clicked.
- Provide simple feedback.

### 2. Basic Toast Example

```
Toast.makeText(
    this,
    "Login successful",
    Toast.LENGTH_SHORT
).show()
```

This displays:

```
Login successful
```

### 3. Toast Duration

#### Toast.LENGTH\_SHORT

Displays the message for a short duration.

```
Toast.makeText(
    this,
    "Saved successfully",
```

```
        Toast.LENGTH_SHORT
    ).show()
```

## Toast.LENGTH\_LONG

Displays the message for a longer duration.

```
Toast.makeText(
    this,
    "Please check your internet connection",
    Toast.LENGTH_LONG
).show()
```

## 4. Practical Example

Suppose a user clicks a **Save** button:

```
saveButton.setOnClickListener {
    Toast.makeText(
        this,
        "Data saved successfully",
        Toast.LENGTH_SHORT
    ).show()
}
```

When the button is clicked, the user receives a temporary confirmation message.

## 5. Intent + Toast Example

Both Intent and Toast can be used together during navigation.

```
loginButton.setOnClickListener {

    Toast.makeText(
        this,
        "Login successful",
        Toast.LENGTH_SHORT
    ).show()

    val intent = Intent(this, HomeActivity::class.java)
    startActivity(intent)
}
```

## Flow

```
User clicks Login
↓
Toast shows "Login successful"
↓
Intent starts HomeActivity
↓
Home screen opens
```

## 6. Key Points of Toast

- Toast provides short, temporary feedback.
- It does not normally require user interaction.
- Toast.makeText() is used to create a Toast.
- .show() displays the Toast.
- LENGTH\_SHORT and LENGTH\_LONG control its duration.
- Toast should be used for simple feedback rather than important actions that require user attention.

## Summary

| Topic  | Purpose                                                 |
|--------|---------------------------------------------------------|
| Intent | Communication and navigation between Android components |

| Topic           | Purpose                                                |
|-----------------|--------------------------------------------------------|
| Explicit Intent | Opens a specific component                             |
| Implicit Intent | Requests an action that another application can handle |
| Intent Extras   | Transfers data between components                      |
| Toast           | Shows a short temporary message to the user            |